Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
про виконання лабораторної роботи №2
«Структури даних та функціональна декомпозиція»
з дисципліни «Професійний Python»

Виконав:
студент 3 курсу
групи ФЕП-33
Зінчук Остап

Викладач:
проф. Свелеба Сергій Андрійович

Львів – 2026

Хід роботи

Тема: Структури даних та функціональна декомпозиція.

Мета: Сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпозувати програму на невеликі функції з чітко визначеною відповідальністю.

Варіант: 4 — Аналіз системи банківських рахунків.

Постановка задачі: Створити модуль обробки структурованих даних банківських рахунків (номер рахунку, ім'я клієнта, баланс, тип рахунку). Реалізувати пошук рахунку за номером, групування рахунків за типом, обчислення загальної суми коштів та середнього балансу, знаходження рахунку з найбільшим/найменшим балансом, фільтрацію за мінімальним балансом, сортування за балансом та формування dict-індексу за номером рахунку. Програма повинна використовувати list, tuple, set, dict, collections (Counter, defaultdict, deque), list/set/dict comprehensions, generator expressions, функції з *args та **kwargs, lambda, closures, decorators. Провести теоретичний аналіз складності та виконати практичний benchmark.

1. Опис та обґрунтування вибору структур даних

• list: Основний масив структурованих записів рахунків. Забезпечує динамічне масштабування та впорядкованість.

• tuple: Незмінюваний фіксований знімок даних ACCOUNTS_TUPLE для захисту цілісності даних.

• set: Збереження унікальних категорій рахунків для миттєвої перевірки приналежності за O(1).

• dict: Представлення об'єктів рахунків та створення швидкого індексу create_account_index за номером рахунку для доступу за O(1).

• collections.Counter: Автоматичний підрахунок кількості рахунків за кожним типом.

• collections.defaultdict: Зручне та ефективне групування рахунків за категорією.

• collections.deque: Двобічна черга для фіксації історії останніх операцій у системі з автоматичним витісненням за O(1).

2. Структура проєкту

```
Lab_02/
├── .gitignore
├── pyproject.toml
├── README.md
├── report/
│   └── Проф. Python-Lab_02-Зінчук.pdf
├── src/
│   └── data_processor/
│       ├── __init__.py
│       ├── data.py
│       ├── decorators.py
│       ├── processors.py
│       ├── analytics.py
│       └── main.py
└── tests/
    └── test_processors.py
```

3. Програмний код створених модулів

Файл src/data_processor/data.py

```
from typing import Any

"""Модуль початкових структурованих даних банківських рахунків."""

accounts: list[dict[str, Any]] = [
    {
        "id": 1,
        "account_number": "UA100001",
        "client_name": "Олена Ткаченко",
        "balance": 15230.50,
        "account_type": "savings",
        "currency": "UAH",
```

```
    },
    {
        "id": 2,
        "account_number": "UA100002",
        "client_name": "Іван Петренко",
        "balance": 4800.00,
        "account_type": "debit",
        "currency": "UAH",
    },
    {
        "id": 3,
        "account_number": "UA100003",
        "client_name": "Марія Коваль",
        "balance": 980.75,
        "account_type": "credit",
        "currency": "UAH",
    },
    {
        "id": 4,
        "account_number": "UA100004",
        "client_name": "Олена Ткаченко",
        "balance": 250.00,
        "account_type": "debit",
        "currency": "UAH",
    },
    {
        "id": 5,
        "account_number": "UA100005",
        "client_name": "Василь Бойко",
        "balance": 42100.00,
        "account_type": "deposit",
        "currency": "UAH",
    },
    {
        "id": 6,
        "account_number": "UA100006",
        "client_name": "Оксана Шевченко",
        "balance": 8750.25,
        "account_type": "savings",
        "currency": "UAH",
    },
    {
        "id": 7,
        "account_number": "UA100007",
        "client_name": "Дмитро Мельник",
        "balance": 3100.00,
        "account_type": "business",
        "currency": "UAH",
    },
    {
        "id": 8,
        "account_number": "UA100008",
        "client_name": "Ірина Бондар",
        "balance": 19500.80,
        "account_type": "deposit",
        "currency": "UAH",
    },
    {
        "id": 9,
        "account_number": "UA100009",
        "client_name": "Тарас Новак",
        "balance": 150.00,
        "account_type": "credit",
        "currency": "UAH",
    },
    {
        "id": 10,
        "account_number": "UA100010",
        "client_name": "Андрій Лисенко",
        "balance": 6420.00,
        "account_type": "business",
        "currency": "UAH",
    },
]

ACCOUNTS_TUPLE: tuple[dict[str, Any], ...] = tuple(accounts)

ACCOUNT_TYPES: set[str] = {acc["account_type"] for acc in accounts}
students = [
```

```
    {
        "id": acc["id"],
        "name": acc["client_name"],
        "group": acc["account_type"],
        "grade": acc["balance"],
    }
    for acc in accounts
]
```

Файл src/data_processor/decorators.py

```
"""Модуль декораторів для логування, заміру часу та повторення."""

from functools import wraps
from time import perf_counter
from typing import Any, Callable


def measure_time(func: Callable[..., Any]) -> Callable[..., Any]:
    """Декоратор для заміру часу виконання функції."""

    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"{func.__name__}: {elapsed:.8f} s")
        return result

    return wrapper


def log_call(func: Callable[..., Any]) -> Callable[..., Any]:
    """Декоратор для логування входу і виходу з функції."""

    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished {func.__name__}")
        return result

    return wrapper


def repeat(
    count: int = 1,
) -> Callable[[Callable[..., Any]], Callable[..., Any]]:
    """Параметризований декоратор (closure) для повторного виклику функції."""

    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            result: Any = None
            for _ in range(count):
                result = func(*args, **kwargs)
            return result

        return wrapper

    return decorator
```

Файл src/data_processor/processors.py

```
"""Модуль обробки та перетворення даних рахунків (processors)."""

from collections import Counter, defaultdict
from collections.abc import Callable
from typing import Any


def get_unique_types(accounts: list[dict[str, Any]]) -> set[str]:
    """Повертає множину унікальних типів рахунків (set comprehension)."""
    return {
        str(acc.get("account_type", acc.get("group", "")))
        for acc in accounts
        if "account_type" in acc or "group" in acc
    }
get_unique_groups = get_unique_types
```

```python
def create_account_index(
    accounts: list[dict[str, Any]],
) -> dict[str, dict[str, Any]]:
    """Створює індекс словника за номером рахунку (dict comprehension)."""
    return {
        str(acc.get("account_number", acc.get("id", ""))): acc
        for acc in accounts
    }


create_student_index = create_account_index


def find_account_by_number(
    accounts: list[dict[str, Any]],
    account_number: str,
) -> dict[str, Any] | None:
    """Лінійний пошук рахунку за номером у списку: складність O(n)."""
    for acc in accounts:
        if (
            acc.get("account_number") == account_number
            or str(acc.get("id")) == account_number
        ):
            return acc
    return None


find_linear = find_account_by_number


def find_account_by_index(
    index: dict[str, dict[str, Any]],
    account_number: str,
) -> dict[str, Any] | None:
    """Швидкий пошук рахунку за індексом у словнику: складність O(1)."""
    return index.get(account_number)


def filter_by_type(
    accounts: list[dict[str, Any]],
    account_type: str,
) -> list[dict[str, Any]]:
    """Фільтрує рахунки за типом (list comprehension)."""
    return [
        acc
        for acc in accounts
        if acc.get("account_type") == account_type
        or acc.get("group") == account_type
    ]


filter_by_group = filter_by_type


def filter_by_min_balance(
    accounts: list[dict[str, Any]],
    min_balance: float,
) -> list[dict[str, Any]]:
    """Фільтрує рахунки за мінімальним балансом (list comprehension)."""
    return [
        acc
        for acc in accounts
        if float(acc.get("balance", acc.get("grade", 0.0))) >= min_balance
    ]


def group_accounts_by_type(
    accounts: list[dict[str, Any]],
) -> dict[str, list[dict[str, Any]]]:
    """Групує рахунки за типом за допомогою collections.defaultdict."""
    result: defaultdict[str, list[dict[str, Any]]] = defaultdict(list)
    for acc in accounts:
        acc_type = str(acc.get("account_type", acc.get("group", "other")))
        result[acc_type].append(acc)
    return dict(result)


group_students = group_accounts_by_type
def count_accounts_by_type(
```

```
    accounts: list[dict[str, Any]],
) -> Counter:
    """Підраховує кількість рахунків кожного типу через collections.Counter."""
    return Counter(
        str(acc.get("account_type", acc.get("group", "other")))
        for acc in accounts
    )


count_students_by_group = count_accounts_by_type


def filter_items(
    items: list[dict[str, Any]],
    predicate: Callable[[dict[str, Any]], bool],
) -> list[dict[str, Any]]:
    """Універсальна функція вищого порядку для фільтрації елементів."""
    return [item for item in items if predicate(item)]


def sort_accounts(
    accounts: list[dict[str, Any]],
    key_func: Callable[[dict[str, Any]], Any] | None = None,
    reverse: bool = False,
) -> list[dict[str, Any]]:
    """Універсальна функція сортування рахунків за переданим ключем."""
    if key_func is None:
        return sorted(
         accounts,
            key=lambda acc: float(acc.get("balance", acc.get("grade", 0.0))),
            reverse=reverse,
        )
    return sorted(accounts, key=key_func, reverse=reverse)


def group_by_nested(
    accounts: list[dict[str, Any]],
    primary_key: str = "account_type",
    secondary_key: str = "currency",
) -> dict[str, dict[str, list[dict[str, Any]]]]:
    """Вкладене групування рахунків за двома характеристиками (Advanced)."""
    grouped: defaultdict[str, defaultdict[str, list[dict[str, Any]]]] = (
        defaultdict(lambda: defaultdict(list))
    )
    for acc in accounts:
        p_val = str(acc.get(primary_key, "unknown"))
        s_val = str(acc.get(secondary_key, "unknown"))
        grouped[p_val][s_val].append(acc)
    return {k: dict(v) for k, v in grouped.items()}
```

Файл src/data_processor/analytics.py

```
"""Модуль аналітики, агрегації, замикань та операцій над рахунками."""

from collections import deque
from collections.abc import Callable
from typing import Any

from .decorators import measure_time


def calculate_total_funds(accounts: list[dict[str, Any]]) -> float:
    """Агрегує загальну суму коштів на всіх рахунках."""
    return sum(
        float(acc.get("balance", acc.get("grade", 0.0)))
        for acc in accounts
    )


@measure_time
def calculate_average_balance(accounts: list[dict[str, Any]]) -> float:
    """Обчислює середній баланс рахунків (генераторний вираз)."""
    if not accounts:
        return 0.0
    return sum(
        float(acc.get("balance", acc.get("grade", 0.0)))
        for acc in accounts
    ) / len(accounts)
calculate_average = calculate_average_balance
```

```python
def find_richest_account(
    accounts: list[dict[str, Any]],
) -> dict[str, Any] | None:
    """Знаходить рахунок з найбільшим балансом за допомогою lambda."""
    if not accounts:
        return None
    return max(
        accounts,
        key=lambda acc: float(acc.get("balance", acc.get("grade", 0.0))),
    )


find_best_student = find_richest_account


def find_poorest_account(
    accounts: list[dict[str, Any]],
) -> dict[str, Any] | None:
    """Знаходить рахунок з найменшим балансом за допомогою lambda."""
    if not accounts:
        return None
    return min(
        accounts,
        key=lambda acc: float(acc.get("balance", acc.get("grade", 0.0))),
    )


def sort_by_balance(
    accounts: list[dict[str, Any]],
    reverse: bool = True,
) -> list[dict[str, Any]]:
    """Сортує рахунки за балансом за спаданням/зростанням через lambda."""
    return sorted(
        accounts,
        key=lambda acc: float(acc.get("balance", acc.get("grade", 0.0))),
        reverse=reverse,
    )


sort_by_grade = sort_by_balance


def calculate_average_values(*values: float) -> float:
    """Обчислює середнє значення позиційних аргументів (*args)."""
    if not values:
        return 0.0
    return sum(values) / len(values)


def create_account_record(**fields: Any) -> dict[str, Any]:
    """Створює запис рахунку з довільних іменованих аргументів (**kwargs)."""
    return dict(fields)


create_record = create_account_record


def create_balance_filter(
    minimum_balance: float,
) -> Callable[[dict[str, Any]], bool]:
    """Створює фільтр замикання (closure), що пам'ятає пороговий баланс."""

    def predicate(account: dict[str, Any]) -> bool:
        return float(
            account.get("balance", account.get("grade", 0.0))
        ) >= minimum_balance

    return predicate


create_grade_filter = create_balance_filter


def create_type_filter(
    target_type: str,
) -> Callable[[dict[str, Any]], bool]:
    """Створює замикання (closure) для фільтрації за типом рахунку."""

    def predicate(account: dict[str, Any]) -> bool:
```

```
        return (
            account.get("account_type") == target_type
            or account.get("group") == target_type
        )

    return predicate


class OperationTracker:
    """Клас збереження історії операцій через collections.deque."""

    def __init__(self, maxlen: int = 10) -> None:
        self.history: deque[str] = deque(maxlen=maxlen)

    def record_operation(self, operation: str) -> None:
        """Додає операцію в історію з автоматичним витісненням."""
        self.history.append(operation)

    def get_recent(self) -> list[str]:
        """Повертає список останніх операцій."""
        return list(self.history
)


def export_summary(accounts: list[dict[str, Any]]) -> dict[str, Any]:
    """Формує підсумковий звіт по рахунках у вигляді словника."""
    total = calculate_total_funds(accounts)
    avg = calculate_average_balance(accounts)
    richest = find_richest_account(accounts)
    poorest = find_poorest_account(accounts)
    return {
        "count": len(accounts),
        "total_funds": total,
        "average_balance": avg,
        "richest_account": richest,
        "poorest_account": poorest,
    }


def pipeline(
    data: list[dict[str, Any]],
    *operations: Callable[[list[dict[str, Any]]], list[dict[str, Any]]],
) -> list[dict[str, Any]]:
    """Виконує ланцюжок трансформацій (Pipeline Processing)."""
    result = data
    for op in operations:
        result = op(result)
    return result
```

Файл src/data_processor/main.py

```
"""Головний модуль демонстрації обробки даних банківської системи."""

import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from data_processor.analytics import (
    OperationTracker,
    calculate_average_balance,
    calculate_average_values,
    calculate_total_funds,
    create_account_record,
    create_balance_filter,
    export_summary,
    find_poorest_account,
    find_richest_account,
    pipeline,
    sort_by_balance,
)
from data_processor.data import (
    ACCOUNTS_TUPLE,
    accounts,
)
from data_processor.decorators import repeat
from data_processor.processors import (
    count_accounts_by_type,
```

```
    create_account_index,
    filter_by_type,
    find_account_by_index,
    find_account_by_number,
    get_unique_types,
    group_accounts_by_type,
    group_by_nested,
)


def print_accounts(title: str, items: list[dict]) -> None:
    """Виводить список банківських рахунків у вигляді таблиці."""
    print(f"\n{title}")
    print("-" * 68)
    print(
        f"{'Номер':12} {'Клієнт':22} {'Тип':12} {'Баланс':>16}"
    )
    print("-" * 68)
    for acc in items:
        num = str(acc.get("account_number", acc.get("id", "")))
        name = str(acc.get("client_name", acc.get("name", "")))
        acc_type = str(acc.get("account_type", acc.get("group", "")))
        bal = float(acc.get("balance", acc.get("grade", 0.0)))
        print(f"{num:12} {name:22} {acc_type:12} {bal:>12.2f} UAH")


@repeat(count=2)
def ping_service() -> str:
    """Демонстрація параметризованого декоратора repeat."""
    return "Сервіс аналітики активний"


def run_benchmark() -> list[tuple[int, float, float]]:
    """Виконує порівняльний benchmark пошуку в list vs dict (Секція 37)."""
    sizes = [1_000, 10_000, 100_000]
    results: list[tuple[int, float, float]] = []

    for size in sizes:
        dataset = [
            {"account_number": f"UA{i:07d}", "balance": float(i)}
            for i in range(size)
        ]
        target_number = f"UA{size - 1:07d}"

        t0 = time.perf_counter()
        _ = find_account_by_number(dataset, target_number)
        t_list = time.perf_counter() - t0

        index = create_account_index(dataset)
        t0 = time.perf_counter()
        _ = find_account_by_index(index, target_number)
        t_dict = time.perf_counter() - t0

        results.append((size, t_list, t_dict))

    return results


def main() -> None:
    """Точка входу програми: повний цикл обробки структурованих даних."""
    print("=" * 68)
    print("ЛАБОРАТОРНА РОБОТА №2: СТРУКТУРИ ДАНИХ ТА ДЕКОМПОЗИЦІЯ")
    print("Варіант 4: Аналіз системи банківських рахунків")
    print("=" * 68)

    print_accounts("Усі банківські рахунки (list of dicts)", accounts)

    print(
        f"\nКількість елементів у кортежі (tuple): {len(ACCOUNTS_TUPLE)}"
    )

    unique_types = get_unique_types(accounts)
    print(f"\nУнікальні типи рахунків (set): {unique_types}")

    print("\n--- Агрегація та замір часу (@measure_time) ---")
    avg_balance = calculate_average_balance(accounts)
    total_funds = calculate_total_funds(accounts)
    print(f"Загальна сума коштів банку: {total_funds:.2f} UAH")
    print(f"Середній баланс: {avg_balance:.2f} UAH")
```

```
    richest = find_richest_account(accounts)
    poorest = find_poorest_account(accounts)
    if richest:
        print(
            f"\nНайбагатший рахунок (lambda max): {richest['account_number']} "
            f"({richest['client_name']}) — {richest['bala
nce']:.2f} UAH"
        )
    if poorest:
        print(
            f"Найбідніший рахунок (lambda min): {poorest['account_number']} "
            f"({poorest['client_name']}) — {poorest['balance']:.2f} UAH"
        )

    ranking = sort_by_balance(accounts, reverse=True)
    print_accounts("Рейтинг рахунків за балансом (сортування lambda)", ranking)

    type_counter = count_accounts_by_type(accounts)
    print("\nКількість рахунків за типом (collections.Counter):")
    for t_name, count in type_counter.items():
        print(f"  • {t_name:12}: {count}")

    grouped = group_accounts_by_type(accounts)
    print("\nГрупування рахунків за типом (collections.defaultdict):")
    for t_name, accs in grouped.items():
        print(f"  • {t_name:12} -> {len(accs)} рахунків")

    nested_grouped = group_by_nested(accounts, "account_type", "currency")
    print(
        f"\nВкладене групування: {len(nested_grouped)} категорій"
    )

    index = create_account_index(accounts)
    search_num = "UA100005"
    found = find_account_by_index(index, search_num)
    print(f"\nПошук за номером {search_num} у dict-index (O(1)):")
    print(f"  {found}")

    threshold = 10_000.0
    min_filter = create_balance_filter(threshold)
    high_balance = [acc for acc in accounts if min_filter(acc)]
    print_accounts(
        f"Рахунки з балансом >= {threshold:.2f} UAH (closure filter)",
        high_balance,
    )

    sample_avg = calculate_average_values(15230.50, 4800.0, 980.75, 42100.0)
    print(f"\nСереднє значення через *args: {sample_avg:.2f}")

    new_account = create_account_record(
        account_number="UA100099",
        client_name="Остап Зінчук",
        balance=75000.00,
        account_type="deposit",
        currency="UAH",
    )
    print(f"\nНовий запис, створений через **kwargs:\n  {new_account}")

    status = ping_service()
    print(f"\nРезультат роботи @repeat(2): {status}")

    tracker = OperationTracker(maxlen=3)
    tracker.record_operation("Пошук UA100005")
    tracker.record_operation("Фільтрація балансу >= 10000")
    tracker.record_operation("Сортування рейтингу")
    tracker.record_operation("Експорт підсумку")
    print(f"\nОстанні 3 операції в deque: {tracker.get_recent()}")

    processed = pipeline(
        accounts,
        lambda d: filter_by_type(d, "savings"),
        lambda d: sort_by_balance(d, reverse=True),
    )
    print(
        f"\nРезультат pipeline (фільтр savings + сортування): "
        f"{len(processed)} записів"
    )

    summary = export_summary(accounts)
```

```
    print(
        f"\nПідсумковий звіт (export_summary):\n"
        f"  Всього: {summary['count']}, "
        f"Фонд: {summary['total_funds']:.2f} UAH, "
        f"Середній: {summary['average_balance']:.2f} UAH"
    )

    print("\n" + "=" * 68)
    print("ЕКСПЕРИМЕНТАЛЬНИЙ BENCHMARK (list search vs dict search)")
    print("=" * 68)
    print(
        f"{'Кількість records':<20} "
        f"{'list search (s)':<22} "
        f"{'dict search (s)':<22}"
    )
    print("-" * 68)
    bench_results = run_benchmark()
    for size, t_l, t_d in bench_results:
        print(f"{size:<20} {t_l:<22.8f} {t_d:<22.8f}")
    print("-" * 68)
    print(
        "Висновок: Пошук у dict (O(1)) працює в сотні разів швидше "
        "за list (O(n)) на великих даних."
    )


if __name__ == "__main__":
    main()
```

Файл tests/test_processors.py

```
"""Модуль автоматизованих тестів для лабораторної роботи №2."""

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))

import unittest
from collections import Counter

from data_processor.analytics import (
    OperationTracker,
    calculate_average_balance,
    calculate_average_values,
    calculate_total_funds,
    create_account_record,
    create_balance_filter,
    create_type_filter,
    export_summary,
    find_poorest_account,
    find_richest_account,
    pipeline,
    sort_by_balance,
)
from data_processor.decorators import (
    log_call,
    measure_time,
    repeat,
)
from data_processor.processors import (
    count_accounts_by_type,
    create_account_index,
    filter_by_min_balance,
    filter_by_type,
    filter_items,
    find_account_by_index,
    find_account_by_number,
    get_unique_types,
    group_accounts_by_type,
    group_by_nested,
    sort_accounts,
)


class TestProcessors(unittest.TestCase):
    """Набір тестів для перевірки функцій обробки структур даних."""

    def setUp(self) -> None:
```

```
        self.sample_accounts = [
            {
                "id": 1,
                "account_number": "UA001",
                "client_name": "Тест 1",
                "balance": 1000.0,
                "account_type": "debit",
                "currency": "UAH",
            },
            {
                "id": 2,
                "account_number": "UA002",
                "client_name": "Тест 2",
                "balance": 5000.0,
                "account_type": "savings",
                "currency": "UAH",
            },
            {
                "id": 3,
                "account_number": "UA003",
                "client_name": "Тест 3",
                "balance": 200.0,
                "account_type": "credit",
                "currency": "USD",
            },
        ]

    def test_get_unique_types(self) -> None:
        types = get_unique_types(self.sample_accounts)
        self.assertIsInstance(types, set)
        self.assertEqual(types, {"debit", "savings", "credit"})

    def test_create_account_index(self) -> None:
        index = create_account_index(self.sample_accounts)
        self.assertIsInstance(index, dict)
        self.assertIn("UA001", index)
        self.assertEqual(index["UA002"]["balance"], 5000.0)

    def test_find_account_by_number(self) -> None:
        found = find_account_by_number(self.sample_accounts, "UA002")
        self.assertIsNotNone(found)
        assert found is not None
        self.assertEqual(found["client_name"], "Тест 2")

    def test_find_account_by_number_not_found(self) -> None:
        found = find_account_by_number(self.sample_accounts, "NON_EXISTING")
        self.assertIsNone(found)

    def test_find_account_by_index(self) -> None:
        index = create_account_index(self.sample_accounts)
        found = find_account_by_index(index, "UA001")
        self.assertIsNotNone(found)
        assert found is not None
        self.assertEqual(found["client_name"], "Тест 1")

    def test_filter_by_type(self) -> None:
        filtered = filter_by_type(self.sample_accounts, "savings")
        self.assertEqual(len(filtered), 1)
        self.assertEqual(filtered[0]["account_number"], "UA002")

    def test_filter_by_min_balance(self) -> None:
        filtered = filter_by_min_balance(self.sample_accounts, 1000.0)
        self.assertEqual(len(filtered), 2)

    def test_group_accounts_by_type(self) -> None:
        gro
uped = group_accounts_by_type(self.sample_accounts)
        self.assertIn("debit", grouped)
        self.assertEqual(len(grouped["debit"]), 1)
        self.assertIn("savings", grouped)

    def test_count_accounts_by_type(self) -> None:
        counts = count_accounts_by_type(self.sample_accounts)
        self.assertIsInstance(counts, Counter)
        self.assertEqual(counts["debit"], 1)
        self.assertEqual(counts["savings"], 1)

    def test_filter_items_higher_order(self) -> None:
        res = filter_items(
```

```
            self.sample_accounts,
            lambda a: a["balance"] > 1000.0,
        )
        self.assertEqual(len(res), 1)
        self.assertEqual(res[0]["account_number"], "UA002")

    def test_sort_accounts_universal(self) -> None:
        res = sort_accounts(self.sample_accounts, reverse=True)
        self.assertEqual(res[0]["account_number"], "UA002")
        self.assertEqual(res[-1]["account_number"], "UA003")

    def test_group_by_nested(self) -> None:
        nested = group_by_nested(
            self.sample_accounts,
            "account_type",
            "currency",
        )
        self.assertIn("debit", nested)
        self.assertIn("UAH", nested["debit"])
        self.assertEqual(len(nested["debit"]["UAH"]), 1)


class TestAnalytics(unittest.TestCase):
    """Набір тестів для модуля аналітики, замикань та агрегації."""

    def setUp(self) -> None:
        self.sample = [
            {
                "account_number": "A1",
                "balance": 100.0,
                "account_type": "debit",
            },
            {
                "account_number": "A2",
                "balance": 500.0,
                "account_type": "savings",
            },
        ]

    def test_calculate_total_funds(self) -> None:
        total = calculate_total_funds(self.sample)
        self.assertEqual(total, 600.0)

    def test_calculate_average_balance(self) -> None:
        avg = calculate_average_balance(self.sample)
        self.assertEqual(avg, 300.0)

    def test_calculate_average_empty(self) -> None:
        self.assertEqual(calculate_average_balance([]), 0.0)

    def test_find_richest_and_poorest(self) -> None:
        richest = find_richest_account(self.sample)
        poorest = find_poorest_account(self.sample)
        self.assertIsNotNone(richest)
        self.assertIsNotNone(poorest)
        assert richest is not None and poorest is not None
        self.assertEqual(richest["account_number"], "A2")
        self.assertEqual(poorest["account_number"], "A1")

    def test_sort_by_balance(self) -> None:
        sorted_accs = sort_by_balance(self.sample, reverse=False)
        self.assertEqual(sorted_accs[0]["account_number"], "A1")
        self.assertEqual(sorted_accs[1]["account_number"], "A2")

    def test_calculate_average_values_args(self) -> None:
        self.assertEqual(calculate_average_values(10.0, 20.0, 30.0), 20.0)
        self.assertEqual(calculate_average_values(), 0.0)

    def test_create_account_record_kwargs(self) -> None:
        rec = create_account_record(
            account_number="A99",
            balance=999.0,
            status="active",
        )
        self.assertEqual(rec["account_number"], "A99")
        self.assertEqual(rec["balance"], 999.0)
        self.assertEqual(rec["status"], "active")

    def test_create_balance_filter_closure(self) -> None:
```

```
        filter_50 = create_balance_filter(50.0)
        filter_200 = create_balance_filter(200.0)
        self.assertTrue(fi
lter_50(self.sample[0]))
        self.assertFalse(filter_200(self.sample[0]))

    def test_create_type_filter_closure(self) -> None:
        filter_debit = create_type_filter("debit")
        self.assertTrue(filter_debit(self.sample[0]))
        self.assertFalse(filter_debit(self.sample[1]))

    def test_operation_tracker_deque(self) -> None:
        tracker = OperationTracker(maxlen=2)
        tracker.record_operation("op1")
        tracker.record_operation("op2")
        tracker.record_operation("op3")
        recent = tracker.get_recent()
        self.assertEqual(len(recent), 2)
        self.assertEqual(recent, ["op2", "op3"])

    def test_export_summary(self) -> None:
        summary = export_summary(self.sample)
        self.assertEqual(summary["count"], 2)
        self.assertEqual(summary["total_funds"], 600.0)
        self.assertEqual(summary["average_balance"], 300.0)

    def test_pipeline_processing(self) -> None:
        result = pipeline(
            self.sample,
            lambda items: [a for a in items if a["balance"] > 200.0],
            lambda items: sort_by_balance(items, reverse=True),
        )
        self.assertEqual(len(result), 1)
        self.assertEqual(result[0]["account_number"], "A2")


class TestDecorators(unittest.TestCase):
    """Набір тестів для декораторів."""

    def test_repeat_decorator(self) -> None:
        counter = 0

        @repeat(count=3)
        def increment() -> int:
            nonlocal counter
            counter += 1
            return counter

        res = increment()
        self.assertEqual(counter, 3)
        self.assertEqual(res, 3)

    def test_measure_time_decorator(self) -> None:
        @measure_time
        def compute() -> int:
            return 42

        self.assertEqual(compute(), 42)

    def test_log_call_decorator(self) -> None:
        @log_call
        def sample() -> str:
            return "ok"

        self.assertEqual(sample(), "ok")


if __name__ == "__main__":
    unittest.main()
```

Файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "data-processor"
version = "0.1.0"
```

```
description = "Laboratory work 2 for Professional Python"
readme = "README.md"
requires-python = ">=3.11"
authors = [
    { name = "Зінчук Остап" }
]

[project.scripts]
data-processor = "data_processor.main:main"
bank-processor = "data_processor.main:main"

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.setuptools.packages.find]
where = ["src"]
```

Файл .gitignore

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
.coverage
.coverage.*
```

4. Результати виконання програми

```
======================================================================
ЛАБОРАТОРНА РОБОТА №2: СТРУКТУРИ ДАНИХ ТА ДЕКОМПОЗИЦІЯ
Варіант 4: Аналіз системи банківських рахунків
======================================================================

Усі банківські рахунки (list of dicts)
----------------------------------------------------------------------
Номер        Клієнт                Тип                      Баланс
----------------------------------------------------------------------
UA100001     Олена Ткаченко        savings           15230.50 UAH
UA100002     Іван Петренко         debit              4800.00 UAH
UA100003     Марія Коваль          credit              980.75 UAH
UA100004     Олена Ткаченко        debit               250.00 UAH
UA100005     Василь Бойко          deposit           42100.00 UAH
UA100006     Оксана Шевченко       savings            8750.25 UAH
UA100007     Дмитро Мельник        business           3100.00 UAH
UA100008     Ірина Бондар          deposit           19500.80 UAH
UA100009     Тарас Новак           credit              150.00 UAH
UA100010     Андрій Лисенко        business           6420.00 UAH

Кількість елементів у кортежі (tuple): 10

Унікальні типи рахунків (set): {'savings', 'debit', 'credit', 'deposit', 'business'}

--- Агрегація та замір часу (@measure_time) ---
calculate_average_balance: 0.00002790 s
Загальна сума коштів банку: 101282.30 UAH
Середній баланс: 10128.23 UAH

Найбагатший рахунок (lambda max): UA100005 (Василь Бойко) — 42100.00 UAH
Найбідніший рахунок (lambda min): UA100009 (Тарас Новак) — 150.00 UAH

Кількість рахунків за типом (collections.Counter):
  • savings     : 2
  • debit       : 2
  • credit      : 2
  • deposit     : 2
  • business    : 2

Пошук за номером UA100005 у dict-index (O(1)):
  {'id': 5, 'account_number': 'UA100005', 'client_name': 'Василь Бойко', 'balance': 42100.0, 'account_type': 'deposit', 'cu

Середнє значення через *args: 15777.81
```

```
Новий запис через **kwargs: {'account_number': 'UA100099', 'client_name': 'Остап Зінчук', 'balance': 75000.0, 'account_type
Результат роботи @repeat(2): Сервіс аналітики активний
Останні 3 операції в deque: ['Фільтрація балансу >= 10000', 'Сортування рейтингу', 'Експорт підсумку']
Підсумковий звіт (export_summary):
  Всього: 10, Фонд: 101282.30 UAH, Середній: 10128.23 UAH
```

5. Аналіз теоретичної складності операцій (Big-O)

Операція	Структура	Складність	Пояснення
Пошук за номером	list	O(n)	Лінійний перебір елементів
Пошук за номером	dict (індекс)	O(1)	Хеш-таблиця за O(1)
Перевірка типу	set	O(1)	Хешування унікальних значень
Групування	defaultdict	O(n)	Один прохід по списку
Сортування	list (Timsort)	O(n log n)	Алгоритм Timsort

6. Результати експериментального Benchmark (Секція 37)

Кількість records	list search (s)	dict search (s)	Прискорення
1 000	0.00007850 s	0.00000130 s	~60x швидше
10 000	0.00092490 s	0.00000170 s	~544x швидше
100 000	0.00779210 s	0.00000400 s	~1948x швидше

Висновок щодо benchmark: Практичні заміри повністю підтвердили теоретичні оцінки складності. Час виконання лінійного пошуку в list зростає пропорційно обсягу даних O(n), тоді як пошук за ключем у dict залишається практично константним O(1), демонструючи колосальне зростання ефективності на великих масивах даних.

7. Відповіді на контрольні запитання (Розділ 42)

1. Що таке list?

Впорядкована, змінювана (mutable) послідовність довільних об'єктів у Python, що дозволяє дублювання елементів та динамічну зміну розміру.

2. Чим list відрізняється від tuple?

list є змінюваним (mutable), тоді як tuple є незмінюваним (immutable). Через це tuple споживає менше пам'яті, швидше ініціалізується і може використовуватись як ключ у словниках чи елемент у множинах.

3. Чому tuple називається immutable?

Тому що після створення кортежу його елементи не можна змінити, додати або видалити.

4. Для чого використовується set?

Для збереження невпорядкованої колекції унікальних елементів, швидкої перевірки належності за O(1), усунення дублікатів та виконання теоретико-множинних операцій.

5. Яка властивість елементів set є важливою?

Елементи множини обов'язково повинні бути незмінюваними та хешованими (hashable), тобто мати реалізований метод __hash__().

6. Що таке dict?

Змінювана структура даних, що зберігає пари «ключ — значення» (key-value) з швидким доступом до значення за ключем у середньому за O(1).

7. Що таке key-value structure?

Структура даних, у якій кожен елемент складається з унікального ідентифікатора (ключа) та пов'язаних з ним даних (значення).

8. Чому пошук key у dict зазвичай швидкий?

Оскільки dict реалізовано на основі хеш-таблиці. Хеш-функція безпосередньо обчислює індекс пам'яті за O(1), усуваючи необхідність лінійного перебору.

9. Що таке hashability?

Властивість об'єкта мати постійне хеш-значення впродовж життя (метод __hash__) та можливість порівняння з іншими об'єктами (__eq__).

10. Чому list не може бути key у dict?

Тому що список є змінюваним (mutable), його вміст може змінюватись, що порушило б цілісність хеш-таблиці. Для списку не реалізовано метод __hash__.

11. Для чого використовується Counter?

Спеціалізований підклас словника з модуля collections, призначений для швидкого та автоматичного підрахунку кількості появ об'єктів у послідовності.

12. Для чого використовується defaultdict?

Підклас словника з collections, який викликає функцію-фабрику за замовчуванням для автоматичної ініціалізації значення, якщо ключа ще немає в словнику.

13. Для чого використовується deque?

Двобічна черга (double-ended queue) з collections, що оптимізована для швидкого додавання та видалення елементів з обох кінців за O(1).

14. Що таке list comprehension?

Компактна синтаксична конструкція створення нового списку: [вираз for елемент in послідовність if умова].

15. Що таке set comprehension?

Конструкція для лаконічного створення нової множини: {вираз for елемент in послідовність if умова}.

16. Що таке dict comprehension?

Конструкція для лаконічного створення словника: {ключ_вираз: знач_вираз for елемент in послідовність if умова}.

17. Що таке generator expression?

Генераторний вираз у круглих дужках (вираз for елемент in послідовність), що повертає об'єкт-генератор з «лінивим» обчисленням по одному елементу без завантаження всієї колекції в пам'ять.

18. Що означає O(1)?

Константна складність — час виконання операції не залежить від розміру вхідних даних n.

19. Що означає O(n)?

Лінійна складність — час виконання зростає прямо пропорційно до кількості елементів n.

20. Що означає O(n log n)?

Лінійно-логарифмічна складність, характерна для ефективних алгоритмів сортування (наприклад, Timsort у Python).

21. Що означає $O(n^2)$?

Квадратична складність — час виконання зростає пропорційно квадрату кількості елементів (наприклад, вкладені цикли).

22. Яка приблизна складність пошуку у list?

O(n), оскільки потрібен послідовний перебір елементів з початку до кінця.

23. Яка середня складність пошуку у set?

O(1) у середньому завдяки обчисленню хешу елемента.

24. Яка середня складність пошуку key у dict?

O(1) у середньому.

25. Чому правильна структура даних може прискорити програму?

Тому що різні структури мають кардинально різну часову складність. Заміна O(n) на O(1) дає скорочення часу обробки в тисячі разів на великих масивах даних.

26. Що таке функціональна декомпозиція?

Розділення складної монолітної задачі на набір невеликих, незалежних функцій з чітко окресленою зоною відповідальності.

27. Які переваги малих functions?

Висока читабельність, простота автономного тестування, легкість повторного використання та зручність супроводу й налагодження.

28. Що таке positional argument?

Аргумент, значення якого зв'язується з параметром функції відповідно до його порядкового номера (позиції) при виклику.

29. Що таке keyword argument?

Іменований аргумент, який передається у вигляді ключ=значення, що дозволяє передавати параметри в довільному порядку.

30. Для чого використовується *args?

Синтаксис у заголовку функції для прийому довільної кількості позиційних аргументів у вигляді кортежу (tuple).

31. Для чого використовується **kwargs?

Синтаксис для прийому довільної кількості іменованих аргументів у вигляді словника (dict).

32. Що таке lambda function?

Анонімна однорядкова функція, визначена ключовим словом lambda, що повертає результат одного виразу.

33. Де доцільно використовувати lambda?

У ролі коротких одноразових функцій-предикатів або функцій ключа (наприклад, аргумент key у sorted, max, min).

34. Чому складну business logic не варто записувати в lambda?

Вона обмежена одним виразом, погіршує читабельність, не підтримує багаторядкові розгалуження та ускладнює діагностику помилок.

35. Що таке closure (замикання)?

Функція, яка запам'ятовує значення змінних зі свого зовнішнього лексичного оточення навіть після завершення роботи зовнішньої функції.

36. Які variables може пам'ятати closure?

Будь-які змінні та параметри, оголошені в обгортаючій зовнішній функції (nonlocal / enclosing scope).

37. Що таке decorator?

Функція вищого порядку, яка приймає іншу функцію як аргумент, розширює або модифікує її поведінку та повертає змінену функцію без зміни її вихідного коду.

38. Для чого застосовуються decorators?

Для логування, заміру продуктивності, автентифікації, кешування (мемоізації) та валідації вхідних даних.

39. Для чого використовується functools.wraps?

Щоб зберігати службові метадані декорованої функції (ім'я __name__, docstring __doc__, анотації) всередині функції-обгортки.

40. Чому потрібно оцінювати складність алгоритму?

Щоб розуміти межі масштабованості системи, попереджати критичні падіння продуктивності та обирати оптимальні алгоритми під великі навантаження.

8. Висновок

У ході виконання лабораторної роботи №2 було здобуто поглиблені практичні навички професійного використання вбудованих та спеціалізованих структур даних Python: list, tuple, set, dict, collections.Counter, collections.defaultdict, collections.deque. Застосовано функціональну декомпозицію для побудови гнучкого модуля обробки структурованих банківських рахунків (Варіант 4). Освоєно синтаксичні конструкції list/set/dict comprehensions та generator expressions, розширені параметри функцій (*args, **kwargs), анонімні функції (lambda), механізм замикань (closures) та фабрик фільтрів, а також реалізовано декоратори моніторингу часу (@measure_time) і параметризовані декоратори повторення (@repeat). Експериментальний benchmark на вибірках до 100 000 записів наочно продемонстрував різницю між лінійним пошуком O(n) та константним хеш-доступом O(1), що підтвердило фундаментальну важливість правильного вибору структур даних для забезпечення високої швидкодії сучасних програмних комплексів.